Coursework 1 Report
Search and Automated Planning

Pinxin Liu Chengtian Gong
OUC CW 1 38

Report date: 28 October 2026

REVIEW DRAFT - video link and final review pending

Table 1: Group details

Member	**HWU student ID**	**HWU username**
Pinxin Liu	H00490337	pl4005
Chengtian Gong	H00490375	cg4005

Video demonstration: PENDING - both members must present; maximum eight minutes.

Reading guide

Part 1 occupies the next three pages; Part 2 occupies the following two pages. The title page and reference list are outside those limits. Measurements refer to the saved runs, not a new timing comparison. This draft requires review by both members.

1 Part 1A: Sudoku as a CSP

Sudoku has 81 variables $X_{r,c}$, one per cell, where r and c range from 1 to 9. An empty cell has domain $\{1, \ldots, 9\}$; a given digit fixes its cell to a singleton domain. Zero denotes an empty input cell and is never a solution value [1-3].

There are 27 AllDifferent constraints: one for each of nine rows, nine columns and nine 3-by-3 boxes. Together with the domains, these require each unit to contain every digit exactly once. A completion must preserve all original givens.

Let m be the number of empty cells. Enumerating every digit assignment gives 9^m complete candidates. Under fixed-size board checking, this yields a coarse $O(9^m)$ upper bound. The unpruned backtracking tree has at most $1 + 9 + \cdots + 9^m$ nodes and the same coarse exponential bound. Counting constraint-checking work separately multiplies this bound by the cost per node. This is a search-size argument for a fixed 9-by-9 board, not an asymptotic hardness result for fixed-size Sudoku [2,4].

Backtracking rejects a partial assignment as soon as it violates a constraint. Later assignments cannot remove an existing duplicate, so every extension of that branch can be skipped. Finding a conflict with k cells unassigned can avoid up to 9^k complete assignments. Pruning usually reduces practical work but does not establish a polynomial worst-case running time.

Minimum remaining values (MRV) selects an empty cell with the smallest current domain [2,4]. The implemented solver recomputes candidates from the current board. It does not maintain propagated domains or enforce arc consistency. Separating those techniques matters when explaining where the measured speed differences arise.

2 Part 1B: Sudoku solver

2.1 Method and implementation

The Python solver uses depth-first backtracking with MRV. The board is a list of nine integer lists. solve() validates and copies its input so the caller retains the original puzzle. TXT and CSV readers accept nine rows with nine digits each; zero denotes a blank. Malformed input and conflicting givens are rejected.

For each empty cell, candidates() removes all values already present in its row, column and box from $\{1, \ldots, 9\}$. At a search node, MRV scans all empty cells. An empty candidate set immediately rejects the branch; otherwise the smallest candidate set determines the next cell. Ties use top-to-bottom, left-to-right order, and candidate digits are tried in ascending order.

An attempted assignment changes one cell. If recursion fails, that cell is reset to zero before the next digit is tried. No maintained domains need restoring. A complete board is accepted; exhausting all legal branches reports no solution. This finite exhaustive search is complete, although difficult inputs can be slow. A row-major option supplies a controlled variable-ordering comparison.

The CLI returns status 0 for a completion, 1 for no solution and 2 for an input error. A separate completion counter stops after two solutions when testing uniqueness; finding one solution alone does not establish uniqueness.

2.1.1 Interface additions

The Tkinter interface provides Load, Solve, Reset and Clear. Load preserves the original puzzle for Reset; Clear removes the board and loaded original. Given digits are black and added digits blue. Duplicate values are highlighted in red, and selecting an empty cell displays its current candidates. Candidate information expresses local legality, not a guarantee of a complete solution. These additions help explain the model without changing the search. Search is synchronous, so expensive inputs can temporarily block the window.

2.1.2 Correctness checks

Seventeen solver tests cover parsing, malformed/conflicting input, solved boards, few blanks, several puzzles, no solution, deterministic behaviour, input preservation and CLI return codes. A separate set-based oracle checks rows, columns, boxes and givens. Six interface-state checks cover conflict marking, candidate display, Clear after manual edits and Reset after solving. The interface tests use widget doubles; native window rendering and event bindings still need manual demonstration.

2.2 Evaluation and theoretical A* analysis

The saved experiment used CPython 3.9.6 on macOS/Darwin 27.0.0 arm64, with five repetitions per puzzle and ordering, and no warm-up. Timing includes the complete solve() call, including validation and copying. File reading, earlier validation, printing and independent solution checks are excluded. The five inputs were constructed locally; no external difficulty labels are claimed.

Table 2: Saved Sudoku measurements (medians over five runs).

Puzzle / ordering	Median ms	Nodes	Backtracks
completed.txt / mrv	0.042	1	0
few_blanks.txt / mrv	0.086	6	0
p1.txt / mrv	2.213	52	0
p2.txt / mrv	27.353	594	537
unsolvable.txt / mrv	0.040	1	0
p1.txt / row-major	1.103	353	301
p2.txt / row-major	1008.228	349303	349246

Nodes count search calls; backtracks count failed placements undone. On p2, MRV visits 594 nodes versus 349,303 for row-major and is about 37 times faster in these runs. On p1, row-major is faster despite more nodes: MRV candidate scans have overhead. These two examples do not establish general superiority. The no-solution case fails immediately and does not test deep unsatisfiable search.

2.2.1 Theoretical A* formulation

A state is a consistent partial board. An action assigns one legal digit to one blank cell at unit cost; a goal is a full valid board. Let h be the number of remaining blanks. It is admissible and consistent: any completion requires exactly one action per remaining blank. With m initial blanks, $g + h = m$ at every reachable state. Thus f cannot distinguish branches, and expansion order depends on tie-breaking. FIFO tie-breaking behaves breadth first and retains an unexpanded frontier that can require exponential memory; depth-first backtracking retains a path. No A* implementation or performance measurement is claimed. Forward checking, AC-3 and exact-cover search are possible alternatives [2,4], but their performance was not measured here.

3 Part 2A/B: Domain and mission design

The reusable lunar domain uses STRIPS and typing [1,5]. Types are rover, lander and waypoint. belongs-to assigns each rover to one lander; lander-at fixes a landing site; connected represents a directed road. aboard represents an undeployed rover, while rover-at gives a deployed rover position. Actions use parameters, rather than hard-coded rover, lander or waypoint identifiers.

Deployment removes aboard and places the rover at its own lander site. Movement requires an existing directed edge and replaces the old rover position. A lander never moves after landing. Well-formed initial ownership and locations are assumed. Single arrows in the supplied maps are not made bidirectional.

One shared memory-free token prevents carrying two unsent data items: taking either an image or scan consumes it. Sending requires the corresponding held data, rover ownership and current position, restores the token and records image-saved or scan-saved at the receiving lander. Sending is wireless; rover and lander do not need to share a waypoint.

A separate sample-free token permits at most one carried sample, independently of data memory. A sample goal is achieved only by store-sample, requiring the rover at its own lander. Storage consumes the lander storage-free token and restores the rover sample slot. No action restores lander storage, so each lander can store at most one sample.

3.1 Mission configuration

Mission 1 fixes Lander 1 at WP1 with Rover 1 aboard and uses the six directed edges of Figure 2. It saves the WP5 image and WP3 scan at Lander 1 and stores the WP1 sample there. Mission 2 starts Rover 1 deployed at WP2 with Lander 1; Lander 2 is fixed at WP1 with Rover 2 aboard. Its nine directed edges follow Figure 3. Lander 1 receives the WP3 image, WP6 scan and WP5 sample; Lander 2 receives the WP2 image, WP4 scan and WP1 sample. Choosing WP1 for Lander 2 is permitted by the specification.

3.2 Base validation

Pyperplan 2.1 with greedy best-first search and hFF found plans of 10 and 20 actions for Missions 1 and 2. Each was independently replayed against the specification rules. Saved base checks compared 2,496 typed action instances in 16 coherent states. A renamed three-rover instance produced a valid 23-action plan. These are targeted tests, not exhaustive proofs or claims of shortest plans.

4 Part 2C: Astronaut extension

The extended domain adds astronaut and room types. astronaut-belongs-to assigns an astronaut to a lander; room-of records room ownership. docking-bay and control-room classify rooms, room-connected describes internal links, and astronaut-at records the current room. Every lander has distinct docking and control rooms, connected in both directions.

The move-astronaut action requires the astronaut at the source, a connecting edge and both rooms belonging to that astronaut's lander. It deletes the old location and adds the new one. Even a supplied cross-lander adjacency cannot permit movement into another lander because room ownership is also checked.

Deployment and sample storage require an astronaut in the receiving lander's docking bay. Both data-transmission actions require an astronaut in its control room. The added astronaut and room parameters are checked for ownership, room role and current presence. Existing rover movement, acquisition, memory and sample rules remain unchanged; no astronaut is needed simply to drive or acquire data.

Mission 3 preserves Mission 2's map, rover/lander setup and all six goals. Alice belongs to Lander 1 and starts in its docking bay; Bob belongs to Lander 2 and starts in its control room. The unchanged base setup and required astronaut facts are verified by the validation tool.

4.1 How the extension changes the plan

The saved 24-action plan contains four astronaut moves. Bob first moves from control room to docking bay, permitting deployment and later sample storage, then returns to the control room for transmissions. Alice moves from docking bay to control room to permit transmissions, then returns to the docking bay before sample storage. Each dependent action is checked against the corresponding lander and room state; astronaut location therefore changes applicability rather than serving as a decorative fact.

4.2 Extension validation and limits

All three missions plan and independently replay successfully. In 23 constructed coherent states, 24,380 typed action instances were checked: 180 allowed instances had the expected effects and 24,200 forbidden instances were rejected. Cases include wrong astronauts, room types, cross-lander movement, co-located landers and inherited capacity rules. Both astronauts move in Mission 3 and every restricted action has a matching astronaut and room. These checks assume valid initial states and do not explore every reachable state.

References

[1] F29AI teaching team. Coursework 1: Search and Automated Planning. 2026. Supplied coursework specification, especially pp. 2, 4-10 and 12-17.

[2] Russell, S. and Norvig, P. Constraint Satisfaction Problems. AIMA teaching slides, public file labelled Chapter 5. https://aima.cs.berkeley.edu/4th-ed/slides-pdf/chapter05.pdf

[3] UC Berkeley CS188. 2.1 Constraint Satisfaction Problems. https://inst.eecs.berkeley.edu/~cs188/textbook/csp/csps.html

[4] UC Berkeley CS188. 2.2 Solving Constraint Satisfaction Problems. https://inst.eecs.berkeley.edu/~cs188/textbook/csp/solving.html

[5] Pyperplan contributors. Pyperplan, version 2.1. https://github.com/aibasel/pyperplan

Experiment sources: reference/part1b/results/benchmark.json and benchmark_rowmajor.json. Planning sources: evidence/base-pddl/ and evidence/part2c/. The repository retains raw plans, traces, source hashes and measurement conditions. The reference entries were retained from the existing contribution draft; no new access date is asserted.